

Pràctiques IIR: Marques fiducials i introducció a OpenCV

Josep Forest

10/2/21



1 Introducció

En aquesta pràctica treballareu i desenvolupareu un estudi dels següents conceptes:

- Marques fiducials
- Marques fiducials ArUco¹

Cal que entengueu i expliqueu els conceptes en el marc de la visió artificial en general, i particularment en l'entorn de la biblioteca **OpenCV**.

A banda dels conceptes (teòrics) que treballareu, en aquesta pràctica també aprendreu a desenvolupar aplicacions de visió artificial amb **OpenCV**, en **C++** i amb **CMake** com a assistent automàtic de desenvolupament.

NOTA: Per realitzar aquesta pràctica cal que tingueu l'entorn de desenvolupament perfectament instal·lat, tal com s'especifica al moodle.

2 Desenvolupant amb OpenCV en C++ i CMake

CMake és un software que és capaç de generar un “build system” de forma automàtica, a partir de comandes organitzades en un fitxer de configuració. Com a tal, és independent del compil·lador utilitzat i és comú combinar CMake amb g++ (a través de l'ordre **make**) o bé amb Visual C++ o MinGW en entorns windows.

El que fa CMake, doncs, és generar tots els fitxers de configuració que requereix el sistema de “build” (sigui quin sigui), per tal que després es pugui compil·lar amb totes les garanties. CMake és especialment útil quan desenvolupem amb múltiples biblioteques de funcions externes, tot i que, en principi, aquí només treballarem amb OpenCV.

2.1 El fitxer CMakeLists.txt per a OpenCV

Moltes biblioteques de funcions incorporen facilitats per tal d'establir fàcilment l'entorn de “build”, i OpenCV no n'és cap excepció. El Llistat 1 mostra el fitxer de configuració que fareu servir per tal de generar l'entorn de “build” per als vostres projectes. El comentem molt ràpidament:

La línia 1 especifica quina versió de **CMake** es fa servir. Actualment és la 3.19. Seguidament, es defineix el nom del projecte (en aquest cas **example**). Després de fixar l'estàndard de llenguatge a C++-11, vé una línia important, la 5, en què hi ha una ordre `find_package(OpenCV REQUIRED)`, que fa exactament això, troba on hi ha la biblioteca OpenCV al nostre sistema i defineix les variables `OPENCV_INCLUDE_DIRS`, `OpenCV_LIBRARY_DIRS` i `OpenCV_LIBRARIES` que seran usats per fixar el(s) directori(s) on es troben els “includes” d'OpenCV, el(s) directori(s) on es troben les biblioteques d'OpenCV (n'hi ha més d'una) i finalment el(s) nom(s) dels fitxers de la(es) pròpia(es) biblioteca(ques) contra les quals s'enllaçarà el vostre projecte.

¹S. Garrido-Jurado, R. Muñoz-Salinas, F. J. Madrid-Cuevas, and M. J. Marín-Jiménez. 2014. “Automatic generation and detection of highly reliable fiducial markers under occlusion”. Pattern Recogn. 47, 6 (June 2014), 2280-2292. DOI=10.1016/j.patcog.2014.01.005

La missió de la resta de línies podeu figurar-vos-la vosaltres mateixos. Si us cal més informació sobre com funciona cmake, podeu referir-vos a cmake.org on podreu trobar tota la informació de les diferents versions de cmake fins avui.

NOTA: Trobareu el fitxer CMakeLists.txt que hem comentat com a material de pràctiques.

```
1 cmake_minimum_required(VERSION 3.1)
2 project(example)
3
4 set (CMAKE_CXX_STANDARD 11)
5 find_package(OpenCV REQUIRED)
6
7 include_directories(${OPENCV_INCLUDE_DIRS})
8 include_directories(${PROJECT_SOURCE_DIR}/include)
9
10 link_directories(${OpenCV_LIBRARY_DIRS})
11
12 set(example_src
13     src/main.cpp
14 )
15 add_executable(example ${example_src})
16 target_link_libraries(example
17     ${OpenCV_LIBRARIES}
18 )
19
20 target_compile_options(example
21     PRIVATE -O3 -std=c++11
22 )
```

Listing 1: Fitxer de configuració “CMakeLists.txt” per projectes d’OpenCV

2.2 Exemple de processat de vídeo amb OpenCV

El codi C++ del llistat 2 mostra un petit programa en C++ que consisteix en un sol “main” i que requereix un paràmetre. El codi està degudament comentat de manera que sigui fàcil entendre què fa cadascuna de les declaracions i funcions que hi intervenen.

Documenteu-vos sobre el que fa cada funció no només a nivell d’ús a través de la documentació de la API a opencv.org, sinó també a nivell conceptual teòric sobre les funcions **cv::GaussianBlur** i **cv::Canny**.

Genereu el “build system” per aquest codi amb cmake i compil·leu segons l’entorn de desenvolupament que hàgiu establert al vostre ordinador. La secció 2.3 dóna una petita guia de com s’aconsegueix compil·lar i enllaçar el programa d’exemple en un entorn linux.

NOTA: Trobareu el fitxer main.cpp que hem comentat com a material de pràctiques.

2.3 Obtenció de l'executable amb CMake i make en Ubuntu

Creeu un directori on hi haurà el codi d'exemple i l'anomeneu, com ara, "example". Dins d'aquest directori hi copieu el fitxer **CMakeLists.txt**. Dins del directori "example", hi creeu un altre directori que es digui "src", i dins d'aquest hi copieu el fitxer **main.cpp**.

En aquest punt, si OpenCV està instal·lat tal com especifica la nota al moodle de l'assignatura, ja tenim tot allò que ens cal per poder obtenir el sistema de "build". Creeu un altre directori anomenat "build", dins el directori "example". Entreu-hi i des d'allà invoqueu cmake de la següent manera:

```
~/example$ mkdir build
~/example$ cd build
~/example$ cmake ..
```

Després d'invocar cmake des de "build", ja tindreu l'entorn creat. Feu un cop d'ull al directori "example" i al directori "build" a veure què hi veieu de nou.

Finalment i per obtenir l'executable només cal que invoqueu **make**. Si no hia hagut cap problema d'instal·lació de res, hauríeu de tenir l'executable al directori "build". Per tal d'invocar el programa, només cal escriure'n el nom de la següent manera:

```
~/examples/build$ ./example 0
```

Això farà córrer el programa amb el dispositiu de vídeo 0, que sol ser la webcam integrada en un portàtil. Mireu quins dispositius de vídeo teniu disponibles tot fent **ls /dev/video***.

3 Binarització de vídeo amb OpenCV

Prenent com a punt de partida l'exemple en C++ anterior, modifiqueu el codi C++ per tal d'obtenir, a més a més de la sortida de contorns Canny, una tercera sortida de vídeo processat que sigui una versió binaritzada simple (amb totes les possibilitats de llindar que ofereix OpenCV) de la imatge de grisos i una quarta sortida de vídeo processat que sigui una versió binaritzada amb llindar adaptatiu.

Expliqueu molt bé què fan les funcions noves que utilitzeu, tant a nivell teòric com a nivell d'ús i comenteu convenientment el nou codi que fareu.

NOTA: Trobareu informació a opencv.org, cerqueu per "thresholding" en general, i mireu també les funcions `cv::threshold` i `cv::adaptiveThreshold` en concret.

```

1  #include "opencv2/core/core.hpp"
2  #include "opencv2/imgproc.hpp"
3  #include "opencv2/highgui.hpp"
4  #include <iostream>
5
6  int main(int argc, char* argv[]) {
7      if (argc < 2){
8          std::cerr << "Usage: " << argv[0] << " <video source no.>" << std::endl;
9          return -1;
10     }
11
12     cv::VideoCapture webCam(std::atoi(argv[1]));          // VideoCapture object
    ↪ declaration. Usually 0 is the integrated, 2 is the first external USB one
13
14     if (webCam.isOpened() == false){ // Check if the VideoCapture object has been
    ↪ correctly associated to the webcam
15         std::cerr << "error: Webcam could not be connected." << std::endl;
16         return -1;
17     }
18
19     cv::Mat imgOriginal;    // input image
20     cv::Mat imgGrayScale;   // grayscale image
21     cv::Mat imgBlurred;     // blurred image
22     cv::Mat imgCanny;       // edge image
23
24     char charCheckForESCKey{0};
25     int lowTh{45};
26     int highTh{90};
27
28     while (charCheckForESCKey != 27 && webCam.isOpened()){ // loop until ESC key
    ↪ is pressed or webcam is lost
29         bool frameSuccess = webCam.read(imgOriginal);      // get next frame
    ↪ from input stream
30
31         if (!frameSuccess || imgOriginal.empty()){          // if the frame was
    ↪ not read or read wrongly
32             std::cerr << "error: Frame could not be read." << std::endl;
33             break;
34         }
35
36         cv::cvtColor(imgOriginal, imgGrayScale, cv::COLOR_BGR2GRAY); // original
    ↪ video is converted to grayscale into imgGrayScale
37         cv::GaussianBlur(imgGrayScale, imgBlurred, cv::Size(5, 5), 1.8); // blurs
    ↪ the grayscale video. Check OpenCV docs for explanation of parameters
38         cv::Canny(imgBlurred, imgCanny, lowTh, highTh);    // Canny edge detection.
    ↪ Check OpenCV docs for explanation of parameters
39
40         // Declaration of windows for output video results. Check OpenCV docs for
    ↪ explanation of parameters
41         cv::namedWindow("imgOriginal", cv::WINDOW_AUTOSIZE);
42         cv::namedWindow("imgCanny", cv::WINDOW_AUTOSIZE);
43
44         // Declaration of trackbars to change thresholds. Check OpenCV docs for
    ↪ explanation of parameters
45         cv::createTrackbar("LowTh", "imgCanny", &lowTh, 100);
46         cv::createTrackbar("HighTh", "imgCanny", &highTh, 100);
47
48         // Show output video results windows
49         cv::imshow("imgOriginal", imgOriginal);
50         cv::imshow("imgCanny", imgCanny);
51
52         charCheckForESCKey = cv::waitKey(1); // gets the key pressed
53     }
54     return 0;
55 }

```
